ASSIGNMENT – 12.5

2303A51655
Batch : 29

Task Description #1 (Sorting – Merge Sort Implementation)

- **Task: Use AI to generate a Python program that implements the Merge Sort algorithm.**
- **Instructions:**
  - **Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.**
  - **Ask AI to include time complexity and space complexity in the function docstring.**
  - **Verify the generated code with test cases.**
- **Expected Output:**
  - **A functional Python script implementing Merge Sort with proper documentation.**

task1.py > ...

```python
def merge(left, right):
        merged.append(left[i])
        i += 1

    # If there are remaining elements in the right list, add them to merged
    while j < len(right):
        merged.append(right[j])
        j += 1

    return merged  # Return the fully merged and sorted list
# Main section to test the merge_sort function
if __name__ == "__main__":
    # Sample lists to test the merge_sort function
    sample_list1 = [38, 27, 43, 3, 9, 82, 10]
    sample_list2 = [5, 2, 9, 1, 5, 6]

    # Print the original lists
    print("Original List 1:", sample_list1)
    print("Original List 2:", sample_list2)

    # Sort the lists using merge_sort and print the results
    sorted_list1 = merge_sort(sample_list1)
    sorted_list2 = merge_sort(sample_list2)

    print("Sorted List 1:", sorted_list1)
    print("Sorted List 2:", sorted_list2)
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py % /usr/bin/python3 "/Users/arepallyramcharan
/ai assistent.py/assignment-12.5.py/task1.py"
Original List 1: [38, 27, 43, 3, 9, 82, 10]
Original List 2: [5, 2, 9, 1, 5, 6]
Sorted List 1: [3, 9, 10, 27, 38, 43, 82]
Sorted List 2: [1, 2, 5, 5, 6, 9]
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py %
```

Observation:

1. AI assistance helps in quickly designing the stack structure and operations.
2. The stack follows the **LIFO (Last In First Out)** principle.
3. Operations like push, pop, peek, and empty-check ensure proper stack functionality.
4. Documentation improves code readability and understanding.
5. AI-generated implementations help beginners understand data structure concepts efficiently.

Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.

task2.py > ...

```python

# Generate a Python program implementing Binary Search.

# Requirements:
# 1. Create a function binary_search(arr, target).
# 2. The function should return the index of the target element.
# 3. If the element is not found, return -1.
# 4. The input list must be sorted.
# 5. Include a detailed docstring explaining:
#    - Binary search concept
#    - Best case time complexity
#    - Average case time complexity
#    - Worst case time complexity
# 6. Add comments explaining every step of the algorithm.
# 7. Include test cases to verify the function.
def binary_search(arr, target):
    """
    Binary Search is an efficient algorithm for finding the position of a target value within a sorted array. I

    Time Complexity:
    - Best Case: O(1) - occurs when the target element is at the middle of the list.
    - Average Case: O(log n) - occurs for random lists.
    - Worst Case: O(log n) - occurs when the target element is not present in the list.

    Parameters:
    arr (list): A sorted list of elements to search through.
    target: The element to search for.

    Returns:
    int: The index of the target element if found; otherwise, -1.
    """

    # Initialize pointers for the left and right ends of the list
    left, right = 0, len(arr) - 1

```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py % /usr/bin/python3 "/Users/arepallyramcharan
/ai assistent.py/assignment-12.5.py/task2.py"
Target 5 found at index: 4
Target 11 found at index: -1
Target 1 found at index: 0
Target 10 found at index: 9
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py %
```

Observation:

- The queue data structure follows the **FIFO (First In First Out)** principle.
- AI guidance simplifies the implementation of enqueue and dequeue operations.
- Front element access helps identify the next element to be removed.
- Size calculation ensures proper monitoring of queue elements.
- AI-assisted development speeds up coding while maintaining correctness.

Task Description #3: Smart Healthcare Appointment Scheduling System
A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:
1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.
Student Task
• Use AI to recommend suitable searching and sorting algorithms.
• Justify the selected algorithms.
• Implement the algorithms in Python.

task3.py > ...

```python

# generate a Python program for a Smart Healthcare Appointment Scheduling System.

# System contains appointment records with:
# appointment_id, patient_name, doctor_name, appointment_time, consultation_fee

# Requirements:
# 1. Recommend suitable algorithms for:
#    - Searching appointment using appointment_id: Linear search, as appointment IDs may not be s
#    - Sorting appointments by time: Timsort (Python's built-in sorted), efficient for general s
#    - Sorting appointments by consultation_fee: Timsort, suitable for numerical sorting.
# 2. Justify the chosen algorithms in 2-3 sentences.
#    Linear search is simple and works for unsorted data with small datasets. Timsort is stable and efficient,
# 3. Implement the searching algorithm in Python.
# 4. Implement the sorting algorithm in Python.
# 5. Use clear functions such as:
#    search_appointment()
#    sort_by_time()
#    sort_by_fee()
# 6. Add docstrings and detailed comments.
# 7. Provide sample appointment data to test the program.

# Sample appointment data

appointments = [
    {"appointment_id": 1, "patient_name": "Alice", "doctor_name": "Dr. Smith", "appointment_time": "2024-07-01
    {"appointment_id": 2, "patient_name": "Bob", "doctor_name": "Dr. Johnson", "appointment_time": "2024-07-01
    {"appointment_id": 3, "patient_name": "Charlie", "doctor_name": "Dr. Smith", "appointment_time": "2024-07-0
    {"appointment_id": 4, "patient_name": "David", "doctor_name": "Dr. Johnson", "appointment_time": "2024-07-0
    {"appointment_id": 5, "patient_name": "Eve", "doctor_name": "Dr. Smith", "appointment_time": "2024-07-01 08
]

def search_appointment(appointments, appointment_id):
    """
    Searches for an appointment by appointment_id using linear search.
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py % /usr/bin/python3 "/Users/arepallyramcharan
/ai assistent.py/assignment-12.5.py/task3.py"
1 08:00', 'consultation_fee': 110}
{'appointment_id': 3, 'patient_name': 'Charlie', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-
07-01 09:00', 'consultation_fee': 120}
{'appointment_id': 4, 'patient_name': 'David', 'doctor_name': 'Dr. Johnson', 'appointment_time': '2024-
07-01 12:00', 'consultation_fee': 130}
{'appointment_id': 2, 'patient_name': 'Bob', 'doctor_name': 'Dr. Johnson', 'appointment_time': '2024-07
-01 11:00', 'consultation_fee': 150}
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py %
```

Observation:

- A singly linked list consists of nodes connected using references.

- Each node contains data and a pointer to the next node.

- Insertion allows dynamic addition of elements in the list.
- Traversal helps display and process elements sequentially.
- AI assistance simplifies the design and implementation of linked lists.

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
pass
```

Expected Output:

• Collision handling using chaining, with well-commented methods.

```python
task4..py > ...

# Generate  a Python program for a Railway Ticket Reservation System.

# Ticket details include:
# ticket_id, passenger_name, train_number, seat_number, travel_date

# Requirements:
# 1. Recommend efficient algorithms for:
#    - Searching tickets using ticket_id
#    - Sorting bookings by travel_date
#    - Sorting bookings by seat_number
# 2. Explain why these algorithms are suitable.
# 3. Implement the searching algorithm.
# 4. Implement the sorting algorithm.
# 5. Add proper function definitions with docstrings.
# 6. Include comments explaining each line.
# 7. Provide example booking records and demonstrate the search and sort operations.
# Sample booking records
bookings = [
    {"ticket_id": 101, "passenger_name": "Alice", "train_number": "123A", "seat_number": "12B", "travel_date":
    {"ticket_id": 102, "passenger_name": "Bob", "train_number": "123A", "seat_number": "12A", "travel_date": "
    {"ticket_id": 103, "passenger_name": "Charlie", "train_number": "456B", "seat_number": "15C", "travel_date
    {"ticket_id": 104, "passenger_name": "David", "train_number": "456B", "seat_number": "15B", "travel_date":
    {"ticket_id": 105, "passenger_name": "Eve", "train_number": "789C", "seat_number": "20A", "travel_date": "
]
def search_ticket(bookings, ticket_id):
    """
    Searches for a ticket by ticket_id using linear search.

    Args:
        bookings (list): List of booking dictionaries.
        ticket_id (int): The ID to search for.

    Returns:
        dict or None: The booking dictionary if found, else None.
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py % /usr/bin/python3 "/Users/arepallyramcharan
/ai assistent.py/assignment-12.5.py/task4..py"
e': '2024-08-01'}
{'ticket_id': 104, 'passenger_name': 'David', 'train_number': '456B', 'seat_number': '15B', 'travel_dat
e': '2024-08-02'}
{'ticket_id': 103, 'passenger_name': 'Charlie', 'train_number': '456B', 'seat_number': '15C', 'travel_d
ate': '2024-08-02'}
{'ticket_id': 105, 'passenger_name': 'Eve', 'train_number': '789C', 'seat_number': '20A', 'travel_date'
: '2024-08-  03'}
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py %
```

Observation:

- A hash table stores data using key-value pairs.
- A hash function converts keys into index positions.
- Collision handling techniques such as chaining improve efficiency.
- Insert, search, and delete operations enable fast data retrieval.
- AI support helps implement hashing techniques accurately.

Task Description #5: Smart Hostel Room Allocation System
A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:
1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.
   Student Task
• Use AI to suggest optimized algorithms.
• Justify the selections.
• Implement the solution in Python.

task5.py > ...

```python

# Genarate Python program for a Smart Hostel Room Allocation System.

# Record fields:
# student_id, room_number, floor, allocation_date

# Requirements:
# 1. Recommend suitable algorithms for:
#    - Searching records using student_id
#    - Sorting records by room_number
#    - Sorting records by allocation_date
# 2. Justify the chosen algorithms.
# 3. Implement the search function.
# 4. Implement the sorting functions.
# 5. Add docstrings explaining algorithm complexity.
# 6. Include detailed comments for each step.
# 7. Test the program with sample hostel allocation data.
# Sample hostel allocation data
allocations = [
    {"student_id": 101, "room_number": "A101", "floor": 1, "allocation_date": "2024-09-01"},
    {"student_id": 102, "room_number": "A102", "floor": 1, "allocation_date": "2024-09-02"},
    {"student_id": 103, "room_number": "B201", "floor": 2, "allocation_date": "2024-09-01"},
    {"student_id": 104, "room_number": "B202", "floor": 2, "allocation_date": "2024-09-03"},
    {"student_id": 105, "room_number": "C301", "floor": 3, "allocation_date": "2024-09-02"},
]
def search_allocation(allocations, student_id):
    """
    Searches for a hostel room allocation record by student_id using linear search.

    Args:
        allocations (list): List of allocation dictionaries.
        student_id (int): The ID to search for.
    Returns:
        dict or None: The allocation dictionary if found, else None.
    """
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py % /usr/bin/python3 "/Users/arepallyramcharan
/ai assistent.py/assignment-12.5.py/task5.py"

Allocations sorted by allocation date:
{'student_id': 101, 'room_number': 'A101', 'floor': 1, 'allocation_date': '2024-09-01'}
{'student_id': 103, 'room_number': 'B201', 'floor': 2, 'allocation_date': '2024-09-01'}
{'student_id': 102, 'room_number': 'A102', 'floor': 1, 'allocation_date': '2024-09-02'}
{'student_id': 105, 'room_number': 'C301', 'floor': 3, 'allocation_date': '2024-09-02'}
{'student_id': 104, 'room_number': 'B202', 'floor': 2, 'allocation_date': '2024-09-03'}
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py %
```

Observation:

1. Graphs represent relationships between nodes using edges.
2. Adjacency lists efficiently store graph connections.
3. Methods like add_vertex and add_edge help construct the graph.
4. Graphs are widely used for network and path representation.
5. AI assistance helps generate structured graph implementations.

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title,

genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.

2. Sort movies based on rating or release year.

Student Task

• Recommend searching and sorting algorithms using AI.

• Justify the chosen algorithms.

• Implement Python functions.

```python

# Generate a Python program simulating an Online Movie Streaming Platform database.

# Movie fields:
# movie_id, title, genre, rating, release_year

# Requirements:
# 1. Recommend suitable algorithms for:
#    - Searching movie by movie_id
#    - Sorting movies by rating
#    - Sorting movies by release_year
# 2. Provide justification for the chosen algorithms.
# 3. Implement the searching algorithm in Python.
# 4. Implement sorting algorithms.
# 5. Use well-structured functions.
# 6. Add docstrings explaining time complexity.
# 7. Include comments explaining the logic.
# 8. Provide example movie records to test the functions.
# Sample movie records
movies = [
    {"movie_id": 1, "title": "Inception", "genre": "Sci-Fi", "rating": 8.8, "release_year": 2010},
    {"movie_id": 2, "title": "The Matrix", "genre": "Action", "rating": 8.7, "release_year": 1999},
    {"movie_id": 3, "title": "Interstellar", "genre": "Sci-Fi", "rating": 8.6, "release_year": 2014},
    {"movie_id": 4, "title": "The Godfather", "genre": "Crime", "rating": 9.2, "release_year": 1972},
    {"movie_id": 5, "title": "Pulp Fiction", "genre": "Crime", "rating": 8.9, "release_year": 1994},
]
def search_movie(movies, movie_id):
    """
    Searches for a movie by movie_id using linear search.

    Time Complexity: O(n) - where n is the number of movies in the list.

    Args:
        movies (list): List of movie dictionaries.
        movie_id (int): The ID to search for.
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py % /usr/bin/python3 "/Users/arepallyramcharan
/ai assistent.py/assignment-12.5.py/task6.py"

Movies sorted by release year:
{'movie_id': 4, 'title': 'The Godfather', 'genre': 'Crime', 'rating': 9.2, 'release_year': 1972}
{'movie_id': 5, 'title': 'Pulp Fiction', 'genre': 'Crime', 'rating': 8.9, 'release_year': 1994}
{'movie_id': 2, 'title': 'The Matrix', 'genre': 'Action', 'rating': 8.7, 'release_year': 1999}
{'movie_id': 1, 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year': 2010}
{'movie_id': 3, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'release_year': 2014}
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py %
```

Observation :

1. Different hospital operations require different data structures.
2. Queue is suitable for handling patient check-in processes.
3. Priority queues help manage emergency cases efficiently.
4. Hash tables allow quick retrieval of patient medical records.

5. Graph structures help represent connections between hospital rooms.

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

• Use AI-assisted reasoning to select algorithms.

• Justify algorithm suitability.

• Implement searching and sorting in Python.

task7.py > ...

```python

# Develop a Python program for a Smart Agriculture Crop Monitoring System.

# Crop record fields:
# crop_id, crop_name, soil_moisture_level, temperature, yield_estimate

# Requirements:
# 1. Select suitable algorithms for:
#    - Searching crop details using crop_id
#    - Sorting crops by soil_moisture_level
#    - Sorting crops by yield_estimate
# 2. Justify the algorithm choices.
# 3. Implement the search function.
# 4. Implement the sorting functions.
# 5. Include docstrings describing algorithm complexity.
# 6. Add line-by-line comments explaining the code.
# 7. Provide sample crop data and demonstrate functionality.
# Sample crop data
crops = [
    {"crop_id": 1, "crop_name": "Wheat", "soil_moisture_level": 30, "temperature": 25, "yield_estimate": 1000},
    {"crop_id": 2, "crop_name": "Corn", "soil_moisture_level": 40, "temperature": 28, "yield_estimate": 1500},
    {"crop_id": 3, "crop_name": "Rice", "soil_moisture_level": 50, "temperature": 30, "yield_estimate": 2000},
    {"crop_id": 4, "crop_name": "Barley", "soil_moisture_level": 20, "temperature": 22, "yield_estimate":
    {"crop_id": 5, "crop_name": "Soybean", "soil_moisture_level": 35, "temperature": 27, "yield_estimate": 120
]
def search_crop(crops, crop_id):
    """
    Searches for a crop by crop_id using linear search.

    Time Complexity: O(n) - where n is the number of crops in the list.

    Args:
        crops (list): List of crop dictionaries.
        crop_id (int): The ID to search for.
    Returns:
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py % /usr/bin/python3 "/Users/arepallyramcharan
/ai assistent.py/assignment-12.5.py/task7.py"
0}
{'crop_id': 5, 'crop_name': 'Soybean', 'soil_moisture_level': 35, 'temperature': 27, 'yield_estimate':
1200}
{'cro                                    at', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 10
00}
{'cro                                        ley', 'soil_moisture_level': 20, 'temperature': 22, 'yield_estimate
': 800}
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py %
```

Navigate to Command
Scroll to Previous Command (⌘↑)
Scroll to Next Command (⌘↓)

Observation:

- Searching crop data using crop ID can be performed efficiently using **Binary Search**.
- Sorting crops by moisture level helps monitor irrigation requirements.
- Sorting by yield estimate helps farmers analyze crop productivity.
- Efficient algorithms improve agricultural data analysis and monitoring.

- AI-assisted algorithm selection helps optimize smart farming systems.

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.

Student Task

• Use AI to recommend algorithms.

• Justify the algorithm selection.

• Implement searching and sorting logic in Python.

task8.py > ...

```python

# Create a Python program for an Airport Flight Management System.

# Flight information fields:
# flight_id, airline_name, departure_time, arrival_time, status

# Requirements:
# 1. Recommend algorithms for:
#    - Searching flight by flight_id
#    - Sorting flights by departure_time
#    - Sorting flights by arrival_time
# 2. Justify the algorithm choices.
# 3. Implement the searching algorithm.
# 4. Implement sorting algorithms.
# 5. Add docstrings explaining algorithm complexity.
# 6. Add comments explaining each step.
# 7. Provide sample flight records and demonstrate searching and sorting.
# Sample flight records
flights = [
    {"flight_id": 101, "airline_name": "Airline A", "departure_time": "2024-08-01 10:00", "arrival_time": "2024
    {"flight_id": 102, "airline_name": "Airline B", "departure_time": "2024-08-01 11:00", "arrival_time": "2024
    {"flight_id": 103, "airline_name": "Airline C", "departure_time": "2024 -08-01 09:00", "arrival_time": "202
    {"flight_id": 104, "airline_name": "Airline D", "departure_time": "2024-08-01 12:00", "arrival_time": "2024
    {"flight_id": 105, "airline_name": "Airline E", "departure_time": "2024-08-01 08:00", "arrival_time": "2024
]
def search_flight(flights, flight_id):
    """
    Searches for a flight by flight_id using linear search.

    Time Complexity: O(n) - where n is the number of flights in the list.

    Args:
        flights (list): List of flight dictionaries.
        flight_id (int): The ID to search for.
    Returns:
```

Output:

```
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py % /usr/bin/python3 "/Users/arepallyramcharan
/ai assistent.py/assignment-12.5.py/task8.py"
'2024-08-01 11:00', 'status': 'On Time'}
{'flight_id': 101, 'airline_name': 'Airline A', 'departure_time': '2024-08-01 10:00', 'arrival_time': '
2024-08-01 12:00', 'status': 'On Time'}
{'flight_id': 102, 'airline_name': 'Airline B', 'departure_time': '2024-08-01 11:00', 'arrival_time': '
2024-08-01 13:00', 'status': 'Delayed'}
{'flight_id': 104, 'airline_name': 'Airline D', 'departure_time': '2024-08-01 12:00', 'arrival_time': '
2024-08-01 14:00', 'status': 'Cancelled'}
arepallyramcharan@Arepallys-MacBook-Air assignment-12.5.py %
```

Observation:

1. Searching flight details using flight ID can be efficiently handled using **Binary Search**.

2. Sorting flights by departure time helps manage flight schedules.
3. Sorting by arrival time improves tracking of incoming flights.
4. Efficient searching and sorting improve airport management systems.
5. AI guidance helps implement optimized algorithms for flight data handling.